

Docker Commands

Управление контейнерами

Команда	Описание
<code>docker-compose up -d</code>	Создает и запускает контейнеры, определенные в <code>docker-compose.yml</code> , в фоновом режиме.
<code>docker-compose down</code>	Останавливает и удаляет контейнеры, сети, тома и образы, созданные <code>docker-compose up</code> .
<code>docker-compose exec <сервис> <команда></code>	Выполняет указанную команду внутри запущенного контейнера сервиса.
<code>docker-compose logs <сервис></code>	Просматривает логи указанного сервиса.
<code>docker ps</code>	Отображает список запущенных контейнеров.
<code>docker ps -a</code>	Отображает список всех контейнеров (включая остановленные).
<code>docker stop <контейнер_id></code>	Останавливает указанный контейнер.
<code>docker start <контейнер_id></code>	Запускает остановленный контейнер.
<code>docker restart <контейнер_id></code>	Перезапускает указанный контейнер.
<code>docker rm <контейнер_id></code>	Удаляет указанный контейнер (должен быть остановлен).

Управление образами

Команда	Описание
<code>docker build -t <имя_образа> <путь_к_dockerfile></code>	Собирает образ из Dockerfile, находящегося по указанному пути, и присваивает ему имя.
<code>docker images</code>	Отображает список доступных образов.
<code>docker rmi <имя_образа></code>	Удаляет указанный образ.

Управление сетями

Команда	Описание
<code>docker network ls</code>	Отображает список доступных сетей Docker.

Команда	Описание
<code>docker network inspect</code> <имя_сети>	Отображает подробную информацию об указанной сети.

Управление томами

Команда	Описание
<code>docker volume ls</code>	Отображает список доступных томов Docker.
<code>docker volume inspect</code> <имя_тома>	Отображает подробную информацию об указанном томе.

Laravel Artisan Commands

Laravel Artisan Commands

Категория	Команда	Описание
-----	-----	-----

Основные	<code>`php artisan serve`</code>	Запускает локальный сервер разработки Laravel.
	<code>`php artisan tinker`</code>	Запускает интерактивную оболочку PHP (REPL) с доступом к окружению Laravel.
Кэширование	<code>`php artisan config:cache`</code>	Создает кэш конфигурации приложения для повышения производительности.
	<code>`php artisan config:clear`</code>	Удаляет кэш конфигурации приложения.
	<code>`php artisan cache:clear`</code>	Удаляет кэш приложения.
	<code>`php artisan view:clear`</code>	Удаляет скомпилированный кэш шаблонов Blade.
База данных	<code>`php artisan migrate`</code>	Запускает все ожидающие миграции базы данных.
	<code>`php artisan migrate:fresh`</code>	Удаляет все таблицы, а затем запускает все миграции заново.
	<code>`php artisan migrate:rollback`</code>	Откатывает последнюю партию миграций.

	`php artisan migrate:status`	Отображает статус всех миграций.
	`php artisan db:seed`	Запускает все или указанные сидеры базы данных.
	`php artisan make:migration <имя>`	Создает новый файл миграции.
	`php artisan make:model <имя> -m`	Создает новую модель Eloquent и соответствующий файл миграции.
	Маршрутизация `php artisan route:list`	Отображает список всех зарегистрированных маршрутов приложения.
	Хранилище `php artisan storage:link`	Создает символическую ссылку из `public/storage` в `storage/app/public`.
	Passport (API) `php artisan passport:keys`	Создает ключи, необходимые для Laravel Passport.
	Разное `php artisan make:<тип> <имя>`	Создает различные компоненты Laravel (контроллеры, модели, middleware, команды и т. д.).
	Например: `php artisan make:controller UserController`.	
	`php artisan optimize:clear`	Очищает все оптимизированные файлы приложения.

Composer Commands

Composer Commands

Категория	Команда	Описание
-----	-----	-----
Установка	`composer install`	Устанавливает зависимости, указанные в файле `composer.lock` (если он существует) или `composer.json`.
	`composer update`	Обновляет зависимости до последних версий, разрешенных в `composer.json`, и обновляет файл `composer.lock`.
	`composer require <пакет>`	Добавляет новый пакет в зависимости проекта и устанавливает его.
	`composer remove <пакет>`	Удаляет пакет из зависимостей проекта и удаляет его.
Управление	`composer show <пакет>`	Отображает информацию об установленном пакете.

	`composer dump-autoload`	Пересоздает файлы автозагрузки (`vendor/autoload.php`). Используется после добавления новых классов.
	`composer clear-cache`	Очищает локальный кэш Composer.
	Скрипты `composer run <скрипт>`	Выполняет скрипт, определенный в секции `scripts` файла `composer.json`.
	Информация `composer info`	Отображает основную информацию о проекте Composer.
	`composer diagnose`	Диагностирует распространенные проблемы конфигурации Composer.
	Обновление `composer self-update`	Обновляет сам Composer до последней версии.

Я надеюсь, этот формат вам подходит! Если вам нужны дополнительные команды или другая категоризация, дайте мне знать.